



US Traffic Accident Severity & Risk Modeling

Final Project Report

Course: CSGY-6513 Big Data, Section C

Semester: Spring 2026

Instructor: Prof. Amit Patel

Team Members:

- Ananya Nadig (an4968)
- Isha Jariwala (ij2221)
- Rhea Shastri (rs9459)

Table Of Contents

Brief Summary.....	3
Problem Statement.....	4
Objectives.....	4
Technologies Used.....	5
Code Execution Instructions.....	6
How to Run the Code.....	6
High-Level Code Logic.....	6
Technological Challenges Faced.....	8
Changes in Technology.....	9
Lessons Learned.....	11
Future Improvements.....	11
Data Sources and Results.....	12
Data Sources.....	12
Dataset: US Accidents Dataset (2016–2023).....	12
Code Repository.....	12
Results.....	12
Key Findings.....	13
Result Screenshots.....	14

Brief Summary

This project presents a large-scale analysis and severity prediction system for US road traffic accidents using the US Accidents Dataset spanning 2016 to 2023, comprising over 7 million records across 49 states. Apache PySpark was used to build a scalable distributed pipeline for data ingestion, cleaning, feature engineering, exploratory data analysis, and machine learning model development. Two classification models - Logistic Regression and Random Forest were developed and compared across multiple imbalance handling strategies. Random Forest with class weighting emerged as the best performing model with an AUC of 0.644, recall of 0.702, and F1 score of 0.397. A novel metric called the State Severity Risk Index was introduced to enable normalized comparison of severity risk across states, revealing Rhode Island and Georgia as the highest risk states with an SSRI of 0.47. The project successfully met all outlined objectives from pipeline development to model comparison and SSRI computation.

Problem Statement

Road traffic accidents represent one of the most pressing public safety concerns in the United States, resulting in significant loss of life, injury, and economic damage every year. Despite the availability of large-scale accident datasets spanning millions of records, extracting meaningful insights from this data remains a substantial challenge due to its sheer volume and complexity.

Traditional data analysis methods are ill-equipped to handle datasets of this magnitude efficiently. Processing millions of records using conventional tools leads to memory limitations, excessive runtimes, and computational bottlenecks that make large-scale analysis impractical. This creates a critical gap between the availability of accident data and the ability to derive actionable insights from it.

Furthermore, accident severity is not determined by any single factor in isolation. It is the result of multiple interacting variables — including time of day, prevailing weather conditions, geographic location, road infrastructure, and visibility — all combining in complex, non-linear ways. Traditional statistical and machine learning methods that rely on simple linear assumptions fail to capture these intricate relationships, resulting in models that are unable to accurately predict or explain severe accident outcomes.

This project addresses these challenges by leveraging big data technologies and advanced machine learning techniques to build a scalable pipeline capable of processing millions of accident records, uncovering complex patterns across temporal, environmental, and geographic dimensions, and developing accurate predictive models for accident severity classification.

Objectives

1. Build a Scalable Distributed Pipeline To design and implement a distributed data processing pipeline using Apache PySpark on Google Cloud Dataproc, capable of ingesting, cleaning, and transforming the US Accidents dataset comprising over 7 million records across 49 US states efficiently and at scale.

2. Analyze Severity Trends (2016–2023) To examine how accident severity has changed over the eight year period from 2016 to 2023, identifying year-wise patterns and understanding the influence of external factors such as the COVID-19 pandemic on traffic accident trends.

3. Identify Environmental and Temporal Risk Factors To analyze the impact of environmental conditions such as weather, temperature, and visibility, as well as temporal factors such as time of day, rush hours, and weekends, on the severity of road traffic accidents.

4. Identify High-Risk Geographic Regions To perform state-level and city-level geographic analysis to identify regions with the highest accident frequency and severity, providing a spatial understanding of accident risk across the United States.

5. Build Machine Learning Models To develop and evaluate Logistic Regression and Random Forest classifiers for predicting accident severity, comparing their performance across multiple imbalance-handling strategies including downsampling and class weighting.

6. Develop the State Severity Risk Index (SSRI) To introduce a normalized risk metric that measures the proportion of high-severity accidents per state, enabling fair comparison of severity risk across states independent of raw accident counts and traffic volume.

7. Interpret Key Risk Factors To analyze feature importance scores and model outputs to identify the most significant factors contributing to accident severity, generating actionable insights for road safety planning and policy decisions.

Technologies Used

The following technologies were used in this project:

Technology	Purpose
PySpark (Spark SQL, Spark MLlib)	Distributed data processing, SQL-style querying, machine learning (LogisticRegression, RandomForestClassifier, VectorAssembler, StandardScaler, StringIndexer, BinaryClassificationEvaluator, MulticlassClassificationEvaluator)
Parquet	Columnar storage format for saving cleaned data with faster read/write performance compared to CSV
Kaggle API	Programmatic download of the US Accidents dataset for reproducibility
Matplotlib / Seaborn	Data visualization for EDA charts, SSRI bar plots, and model comparison charts
Pandas	Local data manipulation for converting Spark DataFrames to pandas for plotting and building the model comparison table
Python	Primary programming language used throughout the project

Code Execution Instructions

The project is implemented as a single end-to-end PySpark pipeline in a Jupyter Notebook. A detailed README describing the code structure and workflow is available here:

README File Link:

<https://github.com/AnanyaSNadig/US-Traffic-Accident-Severity-Risk-Modeling/blob/main/README.md>

How to Run the Code

- Install required dependencies: Python, PySpark, and supporting libraries (matplotlib, seaborn, pandas).
- Set up Kaggle API credentials to download the dataset.
- Open the main notebook: **USAccidents_BigDataProject.ipynb**.
- Run all cells sequentially.

The notebook performs:

- Data ingestion and cleaning
- Feature engineering
- Exploratory data analysis
- SSRI computation
- Model training (Logistic Regression, Random Forest)
- Evaluation and visualization

High-Level Code Logic:

Data Ingestion:

The dataset is downloaded using the Kaggle API and loaded into a PySpark DataFrame. Only the most relevant columns (like severity, time, weather, and road features) are selected to keep the pipeline efficient.

Data Cleaning:

The data is cleaned by removing duplicates and handling missing values. Rows with missing values in important fields like temperature and visibility are dropped. For other columns, simple and practical fixes are used, such as filling pressure with the median and setting missing wind speed to zero. The cleaned data is saved in Parquet format so it can be reused quickly.

Feature Engineering:

New features are created to better capture patterns in the data. This includes time-based features (hour, weekend, rush hour), weather indicators (rain, fog, snow), and a few interaction features.

Categorical values are encoded, and boolean features are converted into numeric form so they can be used in models.

Label Creation:

Accident severity is converted into a binary label, where severity 3 and 4 are treated as high severity. This helps simplify the prediction task.

Exploratory Data Analysis (EDA):

The data is analyzed to understand trends and patterns, such as how accidents vary by time of day, weather conditions, and location. This step also highlights the class imbalance problem and helps guide modeling decisions.

SSRI Computation:

A custom metric called the State Severity Risk Index (SSRI) is calculated to compare states based on the proportion of severe accidents, instead of just total counts.

Model Training:

The data is split into training and testing sets (80/20). Logistic Regression and Random Forest models are trained, with different approaches to handle class imbalance, such as downsampling and class weighting. Logistic Regression uses scaled features, while Random Forest captures more complex patterns.

Evaluation:

Models are evaluated using metrics like AUC, recall, F1 score, and confusion matrices. Threshold tuning is also done for Logistic Regression to improve detection of severe cases.

Result Analysis:

The final results are compared across models, and insights are drawn using visualizations and feature importance to understand what factors contribute most to accident severity.

Technological Challenges Faced

1. Handling Large-Scale Data

One of the biggest challenges was the size of the dataset. With over 7 million records, running operations on the full dataset was slow and often led to memory issues.

To manage this, the dataset was first reduced to around 3 million records and then further sampled to about 1.3 million rows. Intermediate results were cached, and the cleaned dataset was stored in Parquet format. This significantly improved processing speed and made experimentation more manageable.

2. Dealing with Missing and Noisy Data

The dataset had missing values in important columns like temperature, visibility, and weather conditions. Some entries were also inconsistent or noisy.

Rows missing critical values were dropped, while pressure was filled using the median to avoid the effect of outliers. Wind speed was set to zero where it was missing, assuming calm conditions. For categorical columns, “Unknown” was used where needed, and irrelevant entries were filtered out. This helped maintain data quality without introducing bias.

3. High Cardinality in Weather Data

The Weather_Condition column had over 100 unique values, which made it difficult to use directly in the model. Encoding it as-is would have created too many sparse features.

To simplify this, the focus was shifted to extracting meaningful signals. Binary indicators like rain, fog, and snow were created instead. This reduced complexity and made the features more useful for prediction.

4. Class Imbalance

Another major challenge was the imbalance in the dataset. Most accidents were low severity, which caused the initial models to predict almost everything as non-severe.

To address this, different techniques were tried, including 1:1 downsampling, 2:1 downsampling, and class weighting. These approaches helped the models learn patterns for severe accidents more effectively, especially improving recall.

5. Model Training Time

Training models on large datasets, especially Random Forest, took a significant amount of time.

To make this manageable, the model complexity was controlled by limiting the number of trees and depth. Sampling and caching were also used to reduce repeated computations. This made training faster while still maintaining good performance.

6. Feature Scaling for Logistic Regression

Logistic Regression is sensitive to the scale of features, and the dataset had features with very different ranges.

To fix this, all features were standardized before training the model. This helped with stable convergence and improved overall performance.

7. Choosing the Right Threshold

Another challenge was deciding how to convert model probabilities into final predictions. A default threshold of 0.5 did not work well for Logistic Regression due to class imbalance.

So, threshold tuning was done, and a lower threshold (around 0.35) was used to improve detection of severe accidents. For Random Forest, 0.5 worked well and provided a better balance between precision and recall.

Changes in Technology

The core technologies remained consistent with the proposal, including PySpark, Spark MLlib, and Python for distributed processing and modeling.

However, a few practical adjustments were made:

- Development was done in a local/Colab Spark environment instead of full Dataproc for faster iteration.
- Dataset size was reduced (~1.3M rows) to manage runtime.
- Used caching and Parquet for performance optimization.
- Expanded feature engineering (weather indicators, interaction features).

These changes improved efficiency without altering the overall system design.

Uncovered Aspects from Presentations

Due to time limitations during the presentation, a few important aspects of the analysis and modeling could not be explained in detail:

- **Threshold Tuning in Logistic Regression:**
Different probability thresholds were tested (0.25–0.6) instead of using the default 0.5. A

threshold of 0.35 gave better recall for severe accidents, which is important for this use case.

- **Confusion Matrix Analysis:**

Beyond overall metrics, confusion matrices were used to understand model behavior. Random Forest was able to capture around 70% of severe cases, but with some increase in false positives, which is acceptable for a risk-focused problem.

- **Feature Importance (Random Forest):**

The model showed that features like traffic signals, visibility, pressure, and weather indicators had a strong influence on predicting severity.

	Feature	Importance
8	Traffic_Signal	0.521121
13	Pressure	0.200607
2	Wind_Speed	0.118326
7	Junction	0.040669
0	Temperature	0.028993
6	is_weekend	0.023574
3	Hour	0.021878
9	temp_visibility_interaction	0.010797
1	Visibility	0.008862
11	is_rain	0.008644
5	sun_idx	0.006560
12	is_fog	0.006435
10	low_visibility	0.001225
14	is_snow	0.001205
4	is_rush_hour	0.001107

***Figure.** Random Forest feature importance scores showing Traffic_Signal, Pressure and Wind_Speed, as top predictors.*

- **Data Optimization Techniques:**

To handle large data efficiently, caching and Parquet format were used, which significantly reduced runtime and recomputation during experimentation.

Lessons Learned

- One of the biggest realizations was that data preparation matters more than the model itself. Once the data was cleaned properly and better features were added, the results improved a lot without changing the model.
- The class imbalance issue really affected how the models behaved. At first, the model looked accurate, but it was mostly predicting low severity. This made it clear that accuracy alone is not reliable, and metrics like recall and F1 score are more important here.
- There is always a trade-off between precision and recall. Improving detection of severe accidents increased false positives, so the model had to be tuned based on what matters more for the problem.
- Logistic Regression was useful as a baseline to understand the data, but Random Forest performed better because it could capture more complex patterns.
- Working with a large dataset showed that efficiency matters. Techniques like sampling, caching, and using better data formats made a big difference in how quickly the pipeline could run.

Future Improvements

- **Use of More Advanced Models:**
Models like Gradient Boosting or XGBoost could be explored to capture more complex patterns and potentially improve performance.
- **Full Dataset Utilization:**
Running the pipeline on a more powerful cluster would allow use of the complete dataset instead of sampled data, which may improve accuracy.
- **Incorporating Additional Features:**
Adding features like traffic density, road type, or external weather data could provide more context for better predictions.
- **Finer-Grained Geographic Analysis:**
Extending analysis from state-level to city or road-level could help identify more precise high-risk areas.
- **Further Model Tuning:**
Additional hyperparameter tuning and threshold optimization could improve the balance between precision and recall.

Data Sources and Results

Data Sources:

Dataset: US Accidents Dataset (2016–2023)

- **Source:** Kaggle
- **Link:** <https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents>
- **Size:** ~7+ million records (~3 GB)
- **Subset Used:** ~20% sample (~1.3–1.4 million records) for efficient processing

Code Repository

- **GitHub Repository:**
<https://github.com/AnanyaSNadig/US-Traffic-Accident-Severity-Risk-Modeling>

The repository includes the proposal, main notebook, report, presentation, and README with full execution details.

Results

The table below summarizes the performance of all six models on the test set for severe accident detection (Severity ≥ 3):

Model	ROC-AUC	PR-AUC	Precision (Severe)	Recall (Severe)	F1 (Severe)	Notes
Random Forest (weighted)	0.644	0.317	0.276	0.702	0.397	Best overall — highest F1 and AUC
RF 1:1 Downsampling	0.645	0.318	0.282	0.647	0.392	Better precision, fewer false alarms
Baseline LR	0.606	0.274	0.360	0.003	0.005	Failed — predicted everything as low severity

LR 1:1 Downsampling	0.605	0.273	0.235	0.922	0.374	Best recall — catches 92% of severe cases
LR 2:1 Downsampling	0.605	0.273	0.259	0.648	0.370	More conservative tradeoff
Weighted LR	0.605	0.273	0.235	0.922	0.374	Similar to 1:1 but uses full training data

Key Findings

Random Forest with class weighting achieved the best overall balance, with the highest F1 score (0.397) and ROC-AUC (0.644) among all models. It correctly identified approximately 70% of severe accidents while maintaining a reasonable false positive rate.

Logistic Regression with 1:1 downsampling achieved the highest recall (0.922), catching 92% of severe accidents, but at the cost of low precision (0.235) and a large number of false alarms. This variant is suitable for risk-sensitive applications where missing a severe accident is unacceptable.

The Baseline Logistic Regression model completely failed at severe accident detection (recall = 0.003), demonstrating that class imbalance handling is essential for this problem.

The SSRI analysis revealed that states like Rhode Island and Georgia have the highest proportion of severe accidents (~0.47), despite not being the states with the highest total accident counts. This metric provides a normalized view of severity risk that is useful for policy and infrastructure planning.

Result Screenshots

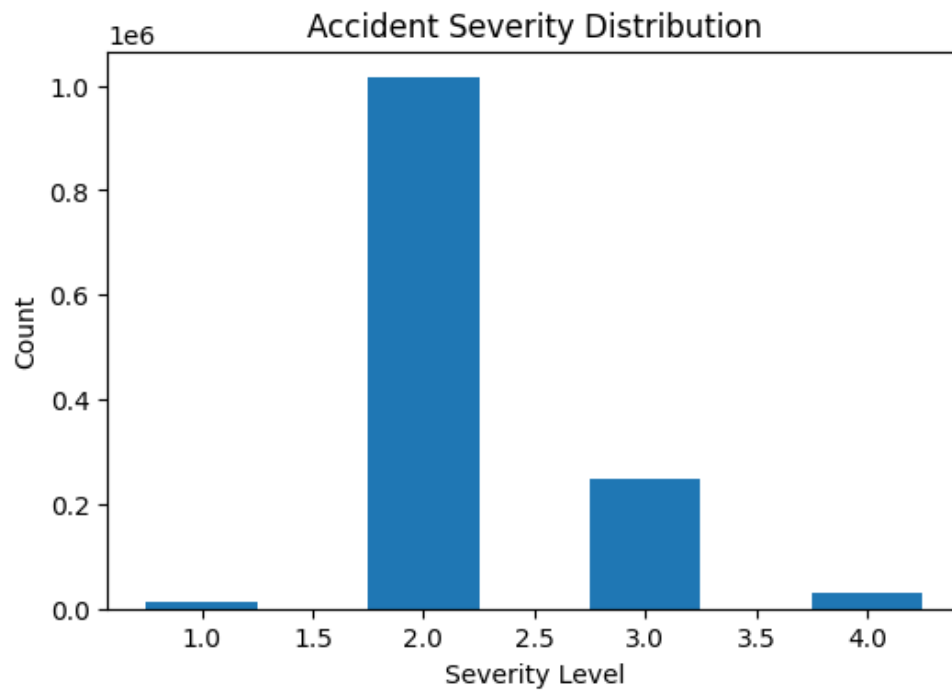


Figure. Distribution of accident severity levels showing heavy class imbalance toward Severity 2

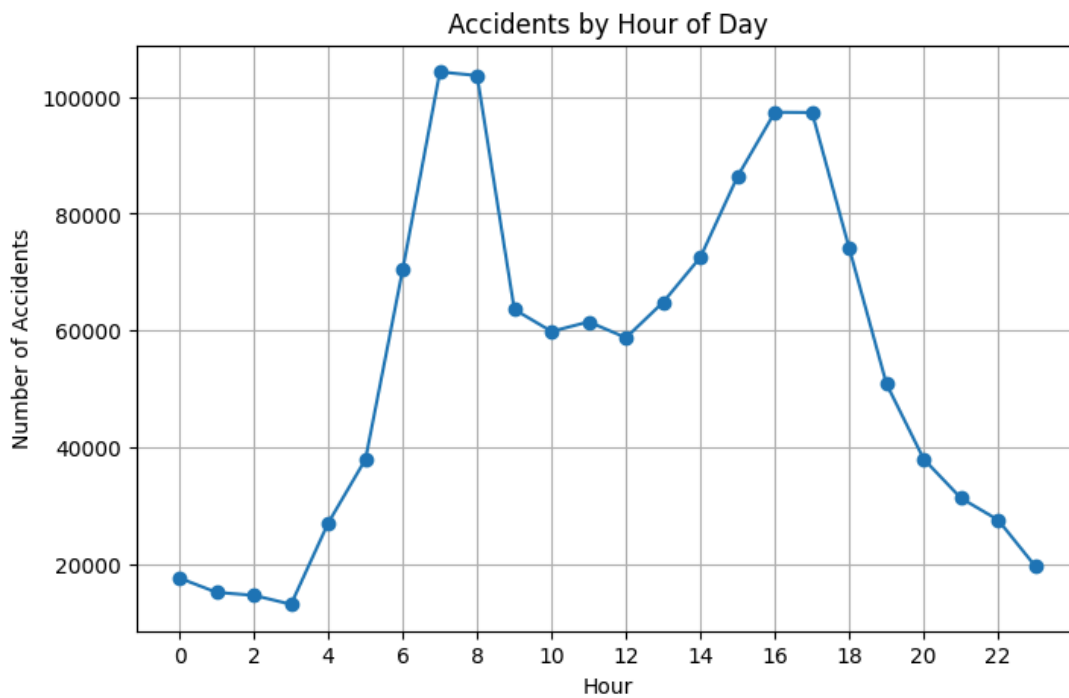


Figure. Hourly accident frequency showing bimodal peaks during morning (7–9 AM) and evening (4–6 PM) rush hours.

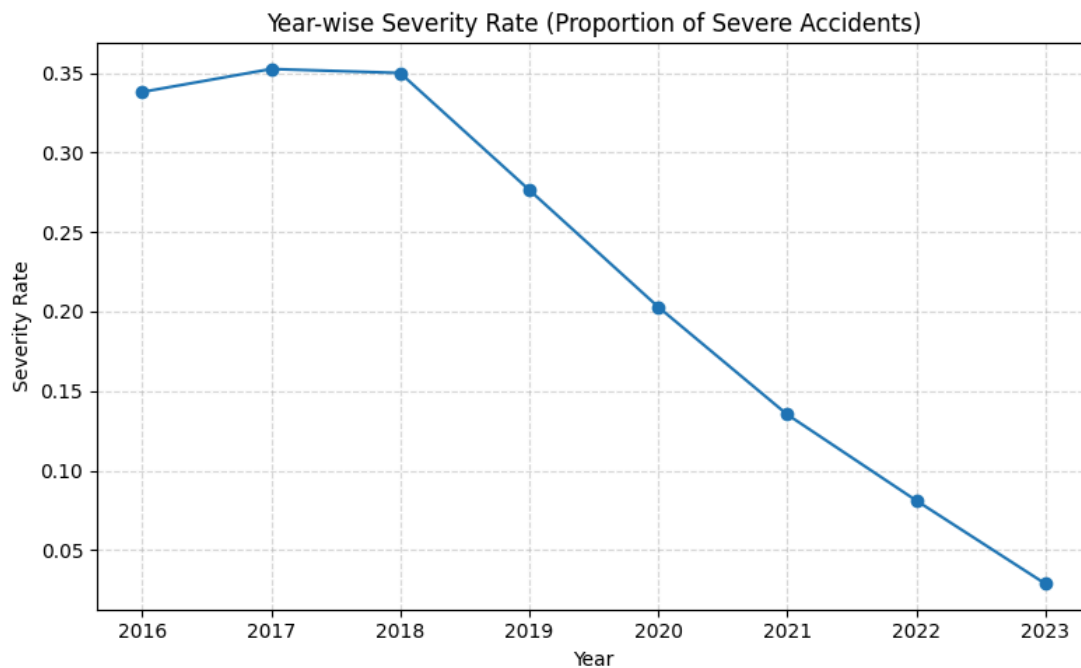


Figure. Year-wise proportion of high-severity accidents showing a declining trend from 35% in 2016 to under 5% in 2023.

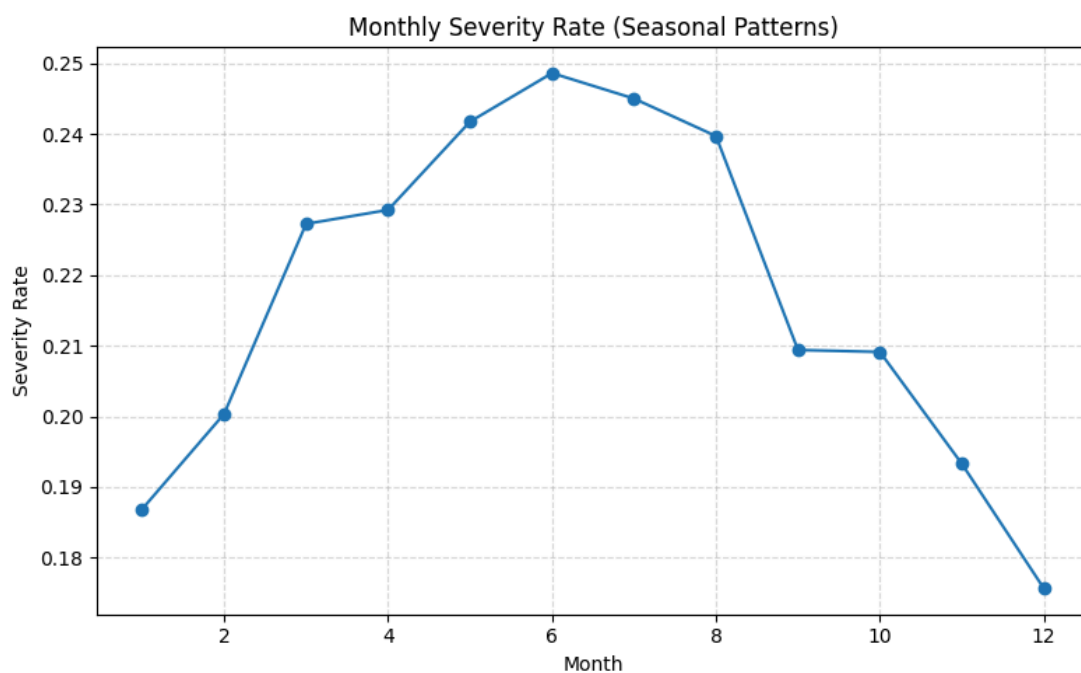


Figure. Monthly severity rate showing seasonal peaks in late spring and early summer.

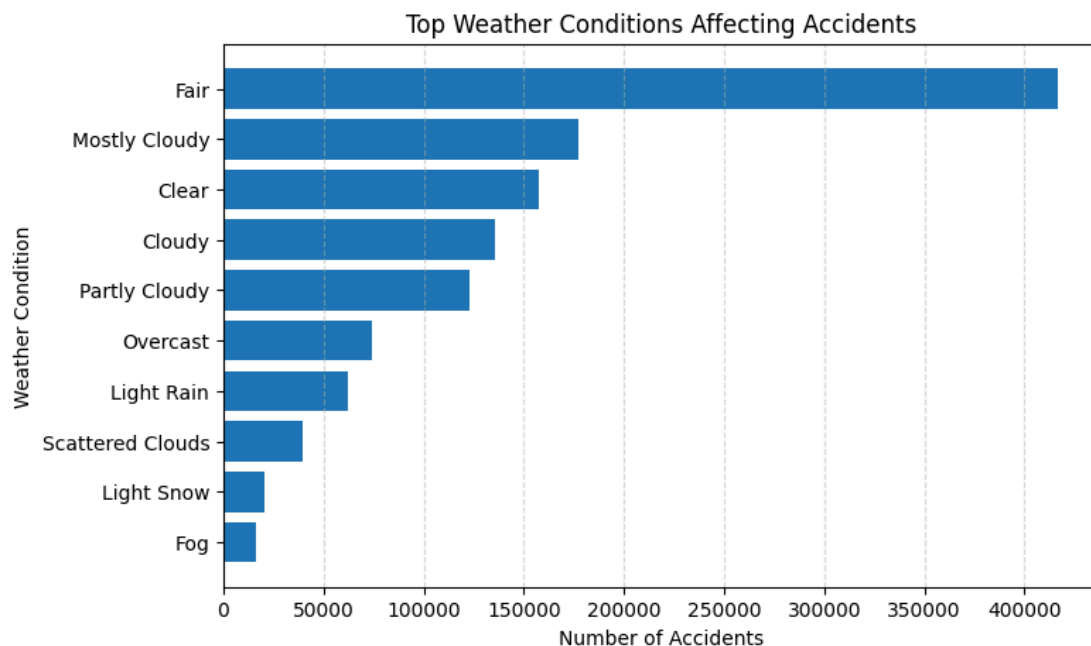


Figure. Top weather conditions by accident count — fair weather dominates due to higher traffic exposure.

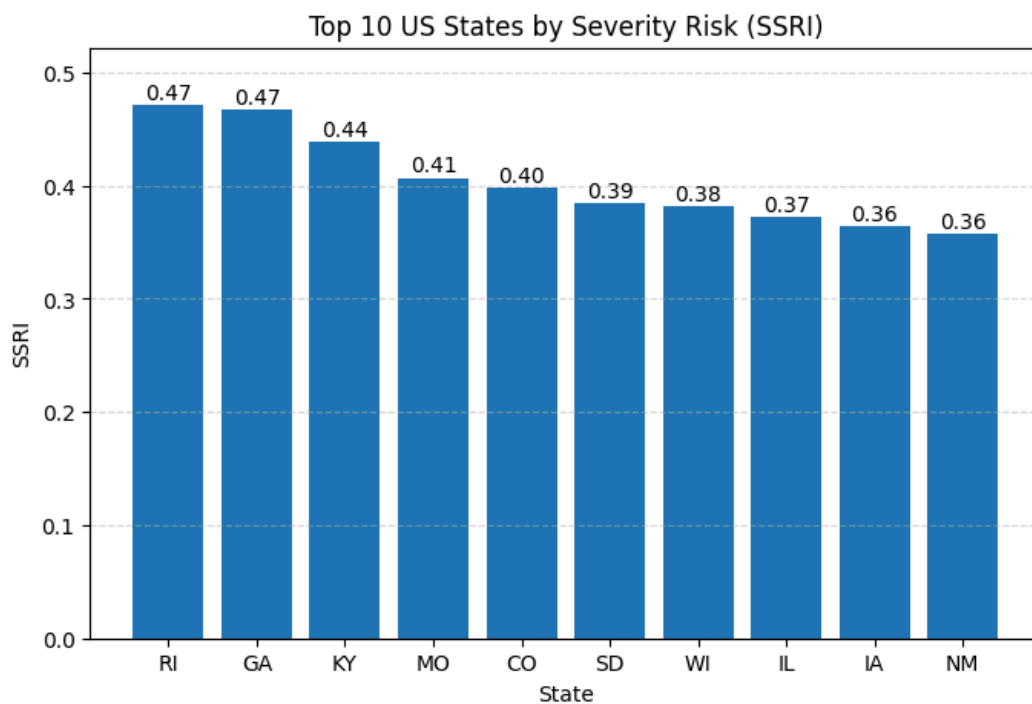


Figure. Top 10 states ranked by State Severity Risk Index, with Rhode Island and Georgia leading at ~0.47.

label	custom_prediction	count
0	0.0	103837
0	1.0	102212
1	0.0	16582
1	1.0	39042

Figure. Random Forest (weighted) confusion matrix — $TP=39,042$, $FP=102,212$, $FN=16,582$, $TN=103,837$.

label	custom_prediction	count
1	0.0	55476
0	0.0	205786
1	1.0	148
0	1.0	263

Figure. Baseline LR confusion matrix showing near-total failure to detect severe accidents ($TP=148$).

	Model	ROC_AUC	PR_AUC	Precision_Severe	Recall_Severe	F1_Severe	Notes
0	Random Forest	0.643945	0.316654	0.276396	0.701891	0.396611	Nonlinear model
1	Random Forest 1:1 Downsampling	0.645235	0.318349	0.281638	0.646627	0.392377	RFC trained on balanced data
2	Baseline LR	0.605984	0.274012	0.360097	0.002661	0.005282	Affected by class imbalance
3	LR 1:1 Downsampling	0.604971	0.272537	0.234729	0.922174	0.374207	Best recall-focused LR model
4	LR 2:1 Downsampling	0.605499	0.273362	0.259146	0.648335	0.370285	More conservative LR model
5	Weighted LR	0.604836	0.272547	0.234724	0.922354	0.374216	Handles imbalance without data loss

Figure. Final comparison table across all six model variants.

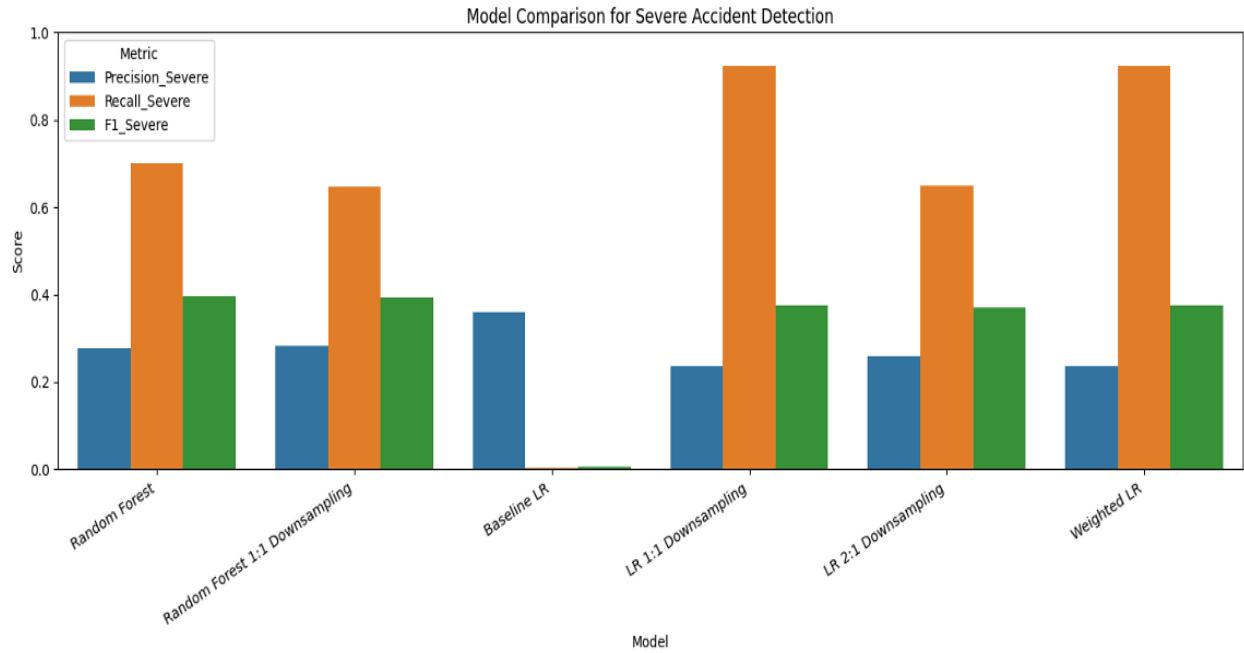


Figure. Precision, Recall, and F1 comparison for severe accident detection across all models.